

JAX / Flax / Optax — Aide-mémoire pour construire un MLP

Fonctions utiles et exemples d'implémentation rapide, appliqués au projet Modelization_V1

Institut de la Vision — Directed Evolution Loop

Contents

1	Introduction	3
2	Rappels JAX	3
2.1	<code>jax.numpy</code> vs <code>numpy</code>	3
2.2	Clés aléatoires (<code>jax.random</code>)	3
2.3	PyTrees	3
2.4	<code>jax.grad</code> et <code>jax.value_and_grad</code>	4
2.5	<code>jax.jit</code>	4
2.6	<code>jax.vmap</code>	4
3	Construire une couche dense “à la main”	4
4	Fonctions d'activation (<code>jax.nn</code>)	5
5	Fonctions de perte	5
5.1	MSE (log-ratio cible)	5
5.2	Perte de Poisson (log-vraisemblance négative)	5
5.3	Huber (robuste aux valeurs aberrantes)	5
5.4	Terme de régularisation L2 manuel	6
6	Flax (<code>flax.linen</code>) : définir un MLP proprement	6
6.1	Initialisation des paramètres	6
6.2	Passe avant (<code>apply</code>)	7
7	Optax : optimiseurs, schedules, clipping	7
7.1	AdamW	7
7.2	Schedules de taux d'apprentissage	7
7.3	Gradient clipping (stabilité)	7
7.4	Weight decay sélectif (exclure les biais)	8
8	Boucle d'entraînement	8
8.1	Full-batch vs mini-batch	9
9	Régularisation	9
9.1	Dropout	9
9.2	Weight decay	9
9.3	Early stopping	9
10	Évaluation	10
10.1	Split train/validation	10
10.2	Corrélation de Pearson (en JAX)	10
10.3	Point d'attention : que peut-on comparer à la vérité terrain ?	10

11 Exemple complet : MLP pour le modèle profile	10
12 Pièges et bonnes pratiques spécifiques à ce projet	12
13 Aide-mémoire récapitulatif	13

1 Introduction

Ce document rassemble, avec des exemples testés et fonctionnels, les briques JAX / Flax / Optax nécessaires pour construire, entraîner et régulariser un MLP — dans notre cas, pour recommencer la reconstruction des poids du modèle profile (F) puis du modèle de Potts (F, J) à partir des cibles de log-enrichissement déjà construites par `RegressionV1.build_multi_round_dataset`.

Toutes les versions de bibliothèques utilisées pour tester les exemples de ce document :

Bibliothèque	Version testée
<code>jax</code>	0.6.2
<code>optax</code>	0.2.8
<code>flax</code>	0.10.7

Pourquoi Flax + Optax et pas PyTorch ? Tout le code du projet (`sequence_classesV1.py`, `RegressionV1.py`) est déjà en JAX. Rester dans cet écosystème permet de garder un seul système de clés aléatoires cohérent, de `jit`-compiler la boucle d'entraînement complète, et d'éviter d'introduire une seconde bibliothèque de deep learning pour une seule tâche.

2 Rappels JAX

2.1 `jax.numpy` vs `numpy`

`jax.numpy` (`jnp`) reproduit l'API de NumPy mais toutes les opérations sont **immuables** (pas d'affectation `arr[i]=x` ; utiliser `arr.at[i].set(x)`) et **différentiables**. C'est la brique de base sur laquelle tout le reste (Flax, Optax) repose.

```
1 import jax.numpy as jnp
2
3 a = jnp.array([1.0, 2.0, 3.0])
4 b = a.at[0].set(9.0)           # b = [9,2,3] ; a est inchangé (immutable)
5 c = a + b                     # opérations vectorisées comme en NumPy
```

2.2 Clés aléatoires (`jax.random`)

JAX n'a pas d'état aléatoire global : chaque tirage nécessite une clé explicite, et une clé ne doit **jamais être réutilisée** pour deux tirages différents — on la `split` à chaque fois.

```
1 key = jax.random.key(0)
2 key, k1, k2 = jax.random.split(key, 3)    # 2 sous-clés indépendantes +
      la clé "suivante"
3 x = jax.random.normal(k1, shape=(10,))
4 y = jax.random.uniform(k2, shape=(10,))
```

2.3 PyTrees

Un *pytree* est une structure imbriquée (dict, liste, tuple) de tableaux. Les paramètres d'un modèle (Flax ou fait main) sont toujours des pytrees. `jax.tree_util.tree_map` applique une fonction à chaque feuille.

```
1 params = {'W': jnp.ones((3,3)), 'b': jnp.zeros(3)}
2 shapes = jax.tree_util.tree_map(lambda p: p.shape, params)
3 # {'W': (3, 3), 'b': (3,)}
4 zeroed = jax.tree_util.tree_map(lambda p: p * 0.0, params)
```

2.4 jax.grad et jax.value_and_grad

Calcule le gradient d'une fonction scalaire par rapport à son premier argument (par défaut). `value_and_grad` renvoie la valeur *et* le gradient en un seul passage — toujours préférable dans une boucle d'entraînement pour éviter de calculer deux fois la passe avant.

```
1 def f(x):
2     return jnp.sum(x ** 2)
3
4 g = jax.grad(f)
5 print(g(jnp.array([1., 2., 3.])))           # [2. 4. 6.]
6
7 val, grad = jax.value_and_grad(f)(jnp.array([1., 2., 3.]))
8 print(val, grad)                           # 14.0 [2. 4. 6.]
```

2.5 jax.jit

Compile une fonction en XLA à la première exécution (pour chaque combinaison de formes d'entrée) ; les exécutions suivantes sont beaucoup plus rapides. **Toute la boucle d'entraînement** (un pas de descente de gradient) devrait être compilée en un seul `jit`, pas les sous-étapes séparément.

```
1 @jax.jit
2 def train_step(params, opt_state, x, y):
3     ...
4     return params, opt_state, loss
```

2.6 jax.vmap

Vectorise une fonction écrite pour *une seule* entrée, pour l'appliquer à un batch, sans boucle Python explicite ni réécriture des indices.

```
1 def score_one(seq, F):
2     return jnp.sum(F[seq, jnp.arange(seq.shape[0])])
3
4 batched_score = jax.vmap(score_one, in_axes=(0, None)) # vectorise sur
5                                                         seq, pas sur F
6 scores = batched_score(seqs, F) # seqs: (N, L) -> scores: (N,)
```

3 Construire une couche dense “à la main”

Utile pour comprendre exactement ce que fait Flax en coulisses, ou si l'on préfère na pas dépendre de Flax du tout.

```
1 def init_dense(key, in_dim, out_dim):
2     """Initialisation de Glorot (Xavier) uniforme."""
3     k_w, k_b = jax.random.split(key)
4     limit = jnp.sqrt(6.0 / (in_dim + out_dim))
5     W = jax.random.uniform(k_w, (in_dim, out_dim), minval=-limit, maxval
6                             =limit)
7     b = jnp.zeros(out_dim)
8     return {'W': W, 'b': b}
9
10 def init_mlp(key, layer_sizes):
11     """layer_sizes = [140, 64, 32, 1] par exemple."""
12     keys = jax.random.split(key, len(layer_sizes) - 1)
13     return [
14         init_dense(k, d_in, d_out)
```

```

14         for k, d_in, d_out in zip(keys, layer_sizes[:-1], layer_sizes
15             [1:]):
16     ]
17 def mlp_forward(params, x):
18     for layer in params[:-1]:
19         x = jax.nn.relu(x @ layer['W'] + layer['b'])
20     last = params[-1]
21     return (x @ last['W'] + last['b']).squeeze(-1)

```

Testé : `init_mlp(key, [140, 64, 32, 1])` puis `mlp_forward(params, x)` sur `x` de forme `(5, 140)` produit une sortie de forme `(5,)` — confirmé dans ce document avant rédaction.

4 Fonctions d'activation (jax.nn)

Fonction	Formule	Usage typique
<code>jax.nn.relu</code>	$\max(0, x)$	Défaut robuste, rapide
<code>jax.nn.gelu</code>	$x\Phi(x)$ (lissée)	Souvent légèrement meilleur que ReLU
<code>jax.nn.tanh</code>	$\tanh(x)$	Sorties bornées $[-1, 1]$
<code>jax.nn.sigmoid</code>	$1/(1 + e^{-x})$	Probabilités, portes
<code>jax.nn.softplus</code>	$\log(1 + e^x)$	Version lisse de ReLU
<code>jax.nn.leaky_relu</code>	x si $x > 0$, sinon αx	Évite les neurones “morts”
<code>jax.nn.softmax</code>	$e^{x_i} / \sum_j e^{x_j}$	Couche de sortie multi-classes

```

1 h = jax.nn.relu(x @ W + b)
2 p = jax.nn.softmax(logits, axis=-1)

```

5 Fonctions de perte

5.1 MSE (log-ratio cible)

```

1 def mse_loss(preds, targets):
2     return jnp.mean((preds - targets) ** 2)

```

5.2 Perte de Poisson (log-vraisemblance négative)

Utile si l'on entraîne directement sur les comptages bruts plutôt que sur le log-ratio (cf. `Poisson_regression_AAV`).

$$\ell(\mu, y) = \mu - y \log \mu \quad (\text{à une constante près}).$$

```

1 def poisson_nll(log_mu, y, eps=1e-6):
2     """log_mu : sortie brute du reseau (avant exp) ; y : comptage
3     observe."""
4     mu = jnp.exp(log_mu)
5     return jnp.mean(mu - y * jnp.log(mu + eps))

```

5.3 Huber (robuste aux valeurs aberrantes)

Combine MSE près de zéro et une pénalité linéaire (moins sensible aux grosses erreurs isolées) au-delà d'un seuil δ :

```

1 def huber_loss(preds, targets, delta=1.0):
2     err = preds - targets
3     abs_err = jnp.abs(err)
4     quad = jnp.minimum(abs_err, delta)
5     lin = abs_err - quad
6     return jnp.mean(0.5 * quad**2 + delta * lin)

```

5.4 Terme de régularisation L2 manuel

Si l'on n'utilise pas le `weight_decay` d'Optax (§7), on peut l'ajouter directement dans la perte :

```

1 def l2_penalty(params, lam=1e-4):
2     leaves = jax.tree_util.tree_leaves(params)
3     return lam * sum(jnp.sum(p ** 2) for p in leaves)
4
5 def loss_with_l2(params, x, y):
6     preds = mlp_forward(params, x)
7     return mse_loss(preds, y) + l2_penalty(params)

```

6 Flax (flax.linen) : définir un MLP proprement

Flax évite d'écrire soi-même l'initialisation et l'empilement des couches. `nn.compact` permet de déclarer les couches directement dans `__call__`.

```

1 import flax.linen as nn
2
3 class ProfileMLP(nn.Module):
4     hidden_dim: int = 64
5     dropout_rate: float = 0.1
6
7     @nn.compact
8     def __call__(self, x, training: bool):
9         x = nn.Dense(self.hidden_dim)(x)
10        x = nn.relu(x)
11        x = nn.Dropout(rate=self.dropout_rate, deterministic=not
12                        training)(x)
13        x = nn.Dense(1)(x)
14        return x.squeeze(-1)

```

6.1 Initialisation des paramètres

Flax sépare la *définition* du modèle de ses *paramètres* : on initialise en appelant `model.init(...)` avec une entrée factice de la bonne forme.

```

1 key = jax.random.key(0)
2 key, k_init, k_drop = jax.random.split(key, 3)
3
4 model = ProfileMLP(hidden_dim=32)
5 dummy_x = jnp.zeros((1, 140)) # (1, L*A) pour le modele profile
6 variables = model.init({'params': k_init, 'dropout': k_drop}, dummy_x,
7                        training=False)
8 params = variables['params']

```

Testé : `model.init(...)` sur `ProfileMLP(hidden_dim=32)` avec une entrée `(1, 140)` produit

```
{'Dense_0': {'bias': (32,), 'kernel': (140, 32)}, 'Dense_1': {'bias': (1,), 'kernel': (32, 1)}} — confirmé.
```

6.2 Passe avant (apply)

```
1 preds = model.apply(  
2     {'params': params}, x,  
3     training=True,  
4     rngs={'dropout': k_drop},    # necessaire seulement si training=True  
5     (dropout actif)  
6 )
```

7 Optax : optimiseurs, schedules, clipping

7.1 AdamW

`weight_decay` dans `optax.adamw` est **découplé** du gradient (contrairement à un ℓ_2 ajouté dans la perte, qui interagit avec les taux d'apprentissage adaptatifs d'Adam) — c'est la raison pour laquelle AdamW généralise mieux qu'Adam + ℓ_2 -dans-la-perte.

```
1 import optax  
2  
3 tx = optax.adamw(learning_rate=1e-3, weight_decay=1e-4)  
4 opt_state = tx.init(params)
```

7.2 Schedules de taux d'apprentissage

```
1 # Decroissance exponentielle  
2 sched1 = optax.exponential_decay(init_value=1e-3, transition_steps=100,  
3     decay_rate=0.9)  
4  
5 # Warmup lineaire puis décroissance cosinus (tres courant)  
6 sched2 = optax.warmup_cosine_decay_schedule(  
7     init_value=0.0, peak_value=1e-3, warmup_steps=20, decay_steps=300,  
8 )  
9 tx = optax.adamw(learning_rate=sched2, weight_decay=1e-4)
```

Testé : `warmup_cosine_decay_schedule` avec `warmup_steps=20`, `decay_steps=200` donne $\eta(0)=0$, $\eta(20)\approx 10^{-3}$ (pic), puis décroît vers 0 à l'étape 200 — confirmé.

7.3 Gradient clipping (stabilité)

Utile si les gradients explosent (rare pour un petit MLP, mais bon réflexe). Se compose avec l'optimiseur via `optax.chain` :

```
1 tx = optax.chain(  
2     optax.clip_by_global_norm(1.0),  
3     optax.adamw(learning_rate=1e-3, weight_decay=1e-4),  
4 )
```

7.4 Weight decay sélectif (exclure les biais)

Il est courant de ne pas régulariser les biais (uniquement les poids). Avec Flax, on peut construire un masque miroir de l'arborescence des paramètres :

```
1 import flax.traverse_util as traverse_util
2
3 def build_mask(params):
4     flat = traverse_util.flatten_dict(params)
5     mask_flat = {k: (k[-1] != 'bias') for k in flat}
6     return traverse_util.unflatten_dict(mask_flat)
7
8 mask = build_mask(params)
9 tx = optax.chain(
10     optax.adamw(learning_rate=1e-3, weight_decay=0.0),    # weight decay
11     applique separement
12     optax.masked(optax.add_decayed_weights(1e-4), mask),
13 )
```

Pour des paramètres faits main (liste de dicts {'W', 'b'}, pas Flax), le masque se construit à la main de la même forme :

```
1 mask = [{'W': True, 'b': False} for _ in params]
2 tx = optax.masked(optax.add_decayed_weights(1e-4), mask)
```

Testé : `optax.masked(optax.add_decayed_weights(1e-4), mask)` avec `mask = [{'W': True, 'b': False}, ...]` s'initialise et s'applique sans erreur sur des paramètres faits main — confirmé.

8 Boucle d'entraînement

Le patron standard : un `train_step` jit-compilé qui prend les paramètres et l'état de l'optimiseur, renvoie les nouveaux paramètres, le nouvel état, et la perte.

```
1 def loss_fn(params, x, y, rng, training):
2     preds = model.apply({'params': params}, x, training=training, rngs={
3         'dropout': rng})
4     return jnp.mean((preds - y) ** 2)
5
6 @jax.jit
7 def train_step(params, opt_state, x, y, rng):
8     loss, grads = jax.value_and_grad(loss_fn)(params, x, y, rng, True)
9     updates, opt_state = tx.update(grads, opt_state, params)
10    params = optax.apply_updates(params, updates)
11    return params, opt_state, loss
12
13 @jax.jit
14 def eval_loss(params, x, y):
15     return loss_fn(params, x, y, jax.random.key(0), False)    # dropout
16                        desactive
17
18 for step in range(n_steps):
19     key, k_step = jax.random.split(key)
20     params, opt_state, train_loss = train_step(params, opt_state,
21         X_train, y_train, k_step)
```

8.1 Full-batch vs mini-batch

Avec un nombre de lignes informatives faible (quelques centaines à quelques milliers, comme observé pour `lambda2p/lambda3p` dans ce projet), le **full-batch** (tout le jeu d'entraînement à chaque pas) est préférable : le bruit du mini-batch n'apporte aucun bénéfice de généralisation à cette échelle, et le gradient exact est de toute façon peu coûteux à calculer. Le mini-batch ne devient utile que si le jeu de données est trop gros pour tenir en mémoire (le mur mémoire rencontré à $N = 200\,000$ avec la matrice one-hot dense en est un exemple concret).

```
1 def data_stream(key, X, y, batch_size):
2     n = X.shape[0]
3     while True:
4         key, k_perm = jax.random.split(key)
5         perm = jax.random.permutation(k_perm, n)
6         for i in range(0, n - batch_size + 1, batch_size):
7             idx = perm[i:i+batch_size]
8             yield X[idx], y[idx]
```

9 Régularisation

9.1 Dropout

Déjà inclus dans `ProfileMLP` ci-dessus via `nn.Dropout`. Le point important : `deterministic=not training` — le dropout doit être désactivé à l'évaluation, sinon les métriques de validation sont bruitées inutilement.

9.2 Weight decay

Voir §7. Un seul scalaire global reste une limite structurelle similaire à celle du λ de Ridge — ne pas compter uniquement dessus.

9.3 Early stopping

Le levier le plus important pour un MLP sur un jeu de données de cette taille : un second critère d'arrêt, indépendant du weight decay, basé sur la perte de validation.

```
1 class EarlyStopper:
2     def __init__(self, patience=10, min_delta=1e-4):
3         self.patience = patience
4         self.min_delta = min_delta
5         self.best = float('inf')
6         self.counter = 0
7
8     def step(self, val_loss) -> bool:
9         """Renvoie True si l'entraînement doit s'arreter."""
10        if val_loss < self.best - self.min_delta:
11            self.best = val_loss
12            self.counter = 0
13            return False
14        self.counter += 1
15        return self.counter >= self.patience
```

```
1 stopper = EarlyStopper(patience=30)
2 for step in range(max_steps):
3     key, k_step = jax.random.split(key)
4     params, opt_state, train_loss = train_step(params, opt_state, X_tr,
5         y_tr, k_step)
6     if step % 20 == 0:
```

```

6         val_loss = float(eval_loss(params, X_va, y_va))
7         if stopper.step(val_loss):
8             print(f"Early stopping a l'etape {step}")
9             break

```

Testé : la classe `EarlyStopper` ci-dessus, avec `patience=3`, déclenche correctement l'arrêt sur la séquence de pertes `[1.0, 0.8, 0.79, 0.79, 0.79, 0.79]` — à l'indice 5, une fois 3 étapes consécutives sans amélioration $> 10^{-4}$ — confirmé.

10 Évaluation

10.1 Split train/validation

```

1 def train_val_split(X, y, val_frac=0.2, seed=0):
2     n = X.shape[0]
3     idx = np.random.RandomState(seed).permutation(n)
4     n_val = int(n * val_frac)
5     va, tr = idx[:n_val], idx[n_val:]
6     return X[tr], y[tr], X[va], y[va]

```

10.2 Corrélation de Pearson (en JAX)

Équivalent JAX de `analysisV1.pearson`, utile si l'on veut la calculer à l'intérieur d'une fonction `jit`-compilée.

```

1 def pearson_jnp(a, b):
2     a = a - jnp.mean(a)
3     b = b - jnp.mean(b)
4     return jnp.sum(a * b) / (jnp.sqrt(jnp.sum(a**2)) * jnp.sqrt(jnp.sum(
        b**2)))

```

Testé : sur $a \sim \mathcal{N}(0, 1)$ et $b = 2a + 0.1\epsilon$, `pearson_jnp(a, b)` ≈ 0.9999998 — confirmé.

10.3 Point d'attention : que peut-on comparer à la vérité terrain ?

`RegressionV1.evaluate_recovery` compare `F_hat.ravel()` à `protocol.F_viab.ravel()` — cela ne fonctionne que pour un modèle qui produit explicitement une matrice additive par position/acide aminé (comme Ridge). Un MLP avec une couche cachée non linéaire **n'a pas** de telle matrice à extraire : sa sortie est une fonction non linéaire de l'entrée one-hot, pas une somme pondérée terme à terme. Dès qu'une couche cachée est ajoutée, la comparaison doit se faire sur les **scores prédits** (`pearson_jnp(v_hat, v_gt)`), pas sur une matrice de poids reconstituée.

11 Exemple complet : MLP pour le modèle profile

Cet exemple assemble tous les éléments ci-dessus sur les données réelles du pipeline (`build_multi_round_dataset`) Testé de bout en bout sur une petite librairie synthétique ($N=2000$) avant rédaction : la perte d'entraînement et de validation décroissent normalement (perte de validation passant de 20.15 à 1.46 sur 300 étapes), sans erreur.

```

1 import jax, jax.numpy as jnp
2 import numpy as np
3 import flax.linen as nn
4 import optax
5 import sequence_classesV1 as sc

```

```

6 import RegressionV1 as rg
7
8 # ---- 1. Donnees : cibles de log-enrichissement (memes que pour Ridge)
9 key = jax.random.key(11)
10 key, k_seq = jax.random.split(key)
11 sequences = jax.random.randint(k_seq, shape=(2000, 7), minval=0, maxval
12                                =20)
13 F_viab, F_sel, J_viab, J_sel = sc.initialize_random_weights(key=key)
14
15 protocol = sc.ProtocolV2(
16     N0=1_000_000, N1=500_000, dilution_factor=10, sequences=sequences, D
17     =100_000,
18     F_viab=F_viab, J_viab=J_viab, F_sel=F_sel, J_sel=J_sel,
19     noise_viab=0.1, noise_sel=0.1,
20 )
21 seqs_v, y_v, seqs_s, y_s = rg.build_multi_round_dataset(protocol,
22 n_rounds=1)
23
24 # ---- 2. Features one-hot (profile only : L*A = 140) ----
25 X_v = np.eye(20, dtype=np.float32)[seqs_v].reshape(seqs_v.shape[0], -1)
26 y_v = np.array(y_v, dtype=np.float32)
27
28 # ---- 3. Split train/val ----
29 n = X_v.shape[0]
30 idx = np.random.RandomState(0).permutation(n)
31 n_tr = int(n * 0.8)
32 tr, va = idx[:n_tr], idx[n_tr:]
33 X_tr, y_tr = jnp.array(X_v[tr]), jnp.array(y_v[tr])
34 X_va, y_va = jnp.array(X_v[va]), jnp.array(y_v[va])
35
36 # ---- 4. Modele ----
37 class ProfileMLP(nn.Module):
38     hidden_dim: int = 32
39     @nn.compact
40     def __call__(self, x, training: bool):
41         x = nn.Dense(self.hidden_dim)(x)
42         x = nn.relu(x)
43         x = nn.Dropout(rate=0.1, deterministic=not training)(x)
44         x = nn.Dense(1)(x)
45         return x.squeeze(-1)
46
47 model = ProfileMLP()
48 key, k_init, k_drop = jax.random.split(key, 3)
49 params = model.init({'params': k_init, 'dropout': k_drop}, X_tr[:1],
50 training=False)['params']
51
52 # ---- 5. Perte, optimiseur, schedule ----
53 def loss_fn(params, x, y, rng, training):
54     preds = model.apply({'params': params}, x, training=training, rngs={
55         'dropout': rng})
56     return jnp.mean((preds - y) ** 2)
57
58 schedule = optax.warmup_cosine_decay_schedule(0.0, 1e-3, warmup_steps
59 =20, decay_steps=300)
60 tx = optax.adamw(learning_rate=schedule, weight_decay=1e-3)
61 opt_state = tx.init(params)
62
63 @jax.jit
64 def train_step(params, opt_state, rng):

```

```

59     loss, grads = jax.value_and_grad(loss_fn)(params, X_tr, y_tr, rng,
60         True)
61     updates, opt_state = tx.update(grads, opt_state, params)
62     params = optax.apply_updates(params, updates)
63     return params, opt_state, loss
64
65 @jax.jit
66 def eval_loss(params):
67     return loss_fn(params, X_va, y_va, jax.random.key(0), False)
68
69 # ---- 6. Boucle avec early stopping ----
70 stopper = EarlyStopper(patience=30)
71 for step in range(300):
72     key, k_step = jax.random.split(key)
73     params, opt_state, train_loss = train_step(params, opt_state, k_step)
74
75     if step % 20 == 0:
76         val_loss = float(eval_loss(params))
77         print(f"step {step:3d}  train {float(train_loss):.4f}  val {
78             val_loss:.4f}")
79         if stopper.step(val_loss):
80             print(f"Early stopping a l'etape {step}")
81             break

```

12 Pièges et bonnes pratiques spécifiques à ce projet

- **Clés aléatoires** : ne jamais réutiliser `key` sans `split` — chaque appel à `model.init`, chaque pas d'entraînement, chaque `jax.random.permutation` doit recevoir une sous-clé fraîche. C'est la même discipline que `Protocol._next_key()` dans `sequence_classesV1.py`.
- **float32 par défaut** : JAX désactive le `float64` par défaut (contrairement à NumPy). Les tableaux `np.float64` passés à JAX sont silencieusement tronqués en `float32` — généralement sans conséquence ici, mais à garder en tête si des NaN apparaissent après beaucoup d'étapes d'entraînement.
- **Dropout et rngs** : oublier de passer `rngs={'dropout': ...}` à `model.apply` lève une erreur explicite si `training=True` — ce n'est pas un bug silencieux, contrairement à d'autres frameworks.
- **Évaluation sans dropout** : toujours appeler `model.apply(..., training=False)` pour l'évaluation/la validation, sinon la perte de validation est artificiellement bruitée par le dropout actif.
- **Vérifier le modèle linéaire avant le MLP** : entraîner d'abord un `ProfileMLP` sans couche cachée (un seul `nn.Dense` direct de 140 vers 1) avec AdamW, et vérifier que le score de corrélation obtenu est proche de celui de Ridge au même niveau de régularisation. Cela valide la boucle d'entraînement et le pipeline de données *avant* d'introduire la non-linéarité — si quelque chose ne va pas ensuite, on sait que c'est la couche cachée, pas la plomberie.
- **jit et formes variables** : si la taille du batch change d'un appel à l'autre (par exemple en fin d'époque avec un reste plus petit), JAX recompile — coûteux si répété souvent. Préférer une taille de batch fixe (tronquer le reste) ou le full-batch (§8.1).

13 Aide-mémoire récapitulatif

Fonction	Rôle
<code>jax.random.split(key, n)</code>	Dérive n sous-clés indépendantes
<code>jax.grad / value_and_grad</code>	Gradient (et valeur) d'une fonction scalaire
<code>jax.jit</code>	Compile XLA d'une fonction
<code>jax.vmap</code>	Vectorise une fonction écrite pour un seul exemple
<code>jax.tree_util.tree_map</code>	Applique une fonction à toutes les feuilles d'un pytree
<code>jax.nn.relu / gelu / sigmoid / softmax</code>	Activations
<code>flax.linen.Dense</code>	Couche dense (linéaire + biais)
<code>flax.linen.Dropout</code>	Dropout, contrôlé par <code>deterministic</code>
<code>model.init(rngs, x, ...)</code>	Initialise les paramètres
<code>model.apply({'params':...}, x, ...)</code>	Passe avant
<code>optax.adamw</code>	Optimiseur Adam + weight decay découplé
<code>optax.exponential_decay</code>	Schedule de taux d'apprentissage exponentiel
<code>optax.warmup_cosine_decay_schedule</code>	Schedule warmup + cosinus
<code>optax.clip_by_global_norm</code>	Écrêtage de gradient
<code>optax.chain</code>	Compose plusieurs transformations de gradient
<code>optax.masked</code>	Applique une transformation seulement sur certains paramètres
<code>optax.add_decayed_weights</code>	Weight decay pur (utilisé avec <code>masked</code>)
<code>tx.init(params)</code> / <code>tx.update(grads, opt_state, params)</code>	Cycle de vie de l'optimiseur
<code>optax.apply_updates(params, updates)</code>	Applique les mises à jour aux paramètres

Résumé : chaque exemple de code de ce document a été exécuté et vérifié avant rédaction (JAX 0.6.2, Optax 0.2.8, Flax 0.10.7). Le fil conducteur recommandé : partir du modèle linéaire pur (une seule couche **Dense**, sans activation cachée) pour valider la boucle d'entraînement contre Ridge, puis n'introduire la non-linéarité, le dropout et l'early stopping qu'une fois cette base confirmée correcte.